

DiabetesCare 360 — Backend Build Brief for Google Antigravity

This is a mission brief to hand to an Antigravity agent (or split across several agents/missions). It builds the backend defined in the DiabetesCare 360 PRD: patient data APIs, IoT ingestion, an ML forecasting service, an NLP dietary service, real-time alerts, and a clinician dashboard API.

How to use this file: Run the phases in order. Do not start a phase until the previous phase's "Definition of Done" checks pass — Antigravity's terminal and browser surfaces should be used to actually run those checks (start the server, curl the endpoint, run the test suite), not just to write code that looks correct. If a phase fails verification, fix it before moving on; later phases assume earlier ones actually work.

Tech stack (fixed — don't substitute mid-project)

Layer	Choice	Why
Core API	Node.js + TypeScript + Fastify (or Express)	Matches PRD's Node.js/FastAPI note; TypeScript catches schema drift early
ML & NLP service	Python + FastAPI	PRD calls for LSTM/Random Forest + NLP; Python has the ML ecosystem
Primary database	PostgreSQL 15+ with the TimescaleDB extension	Relational data (users, meds, meals) + efficient time-series storage for glucose readings
Cache / pub-sub	Redis	Session cache, rate limiting, and pub-sub for real-time alerts
Auth	JWT (access + refresh tokens), bcrypt for password hashing	Stateless, works across mobile + web + clinician dashboard
Real-time transport	WebSockets (Socket.io or native ws)	Push glucose updates and risk alerts to open clients
Containerization	Docker + docker-compose for local dev	Antigravity can spin up the whole stack with one command and verify it
Testing	Jest (Node service), Pytest (Python service), Supertest for HTTP	Needed for each phase's Definition of Done

Two services talk to each other over REST: `(api)` (Node) owns all patient/clinician data and auth; `(ml-service)` (Python) is stateless and only does inference/training, called internally by `(api)`. Do not let the frontend call `(ml-service)` directly — everything goes through `(api)` so auth and access control stay in one place.

Phase 0 — Project Scaffold & Environment

Goal: A runnable skeleton with both services, the database, and Redis wired together — before any real feature exists.

1. Create a monorepo: `(/api)` (Node/TS), `(/ml-service)` (Python/FastAPI), `(/docker-compose.yml)`, `(/docs)`.
2. `(docker-compose.yml)` with services: `(postgres)` (with TimescaleDB image), `(redis)`, `(api)`, `(ml-service)`. Use named volumes for Postgres data so restarts don't wipe dev data.
3. `(.env.example)` in the repo root listing every variable both services need (DB URL, JWT secrets, Redis URL, ML service internal URL) — never commit a real `(.env)`.
4. In `(/api)`: initialize TypeScript, Fastify, a `(/health)` route returning `{ status: "ok" }`.
5. In `(/ml-service)`: initialize FastAPI, a `(/health)` route returning `{ status: "ok" }`.
6. Add root `(README.md)` explaining how to run `(docker-compose up)` and hit both health checks.

Definition of Done (verify, don't assume):

- `(docker-compose up)` brings up all four containers with no crash-loop.
 - `(curl localhost:<api-port>/health)` and `(curl localhost:<ml-port>/health)` both return 200.
 - `(docker-compose down && docker-compose up)` again still works (volumes persist correctly).
-

Phase 1 — Database Schema & Models

Goal: Every entity the PRD's functional requirements need, with migrations (not hand-edited SQL) so schema changes are tracked.

Use a migration tool (Prisma or Knex/Drizzle for the Node side). Core tables:

- `(users)` — id, role (`(patient)` | `(caregiver)` | `(clinician)`), email, password_hash, name, diabetes_type (nullable), created_at
- `(care_links)` — links a patient to a caregiver or clinician, with a sharing status (pending/active/revoked)
- `(glucose_readings)` (TimescaleDB hypertable) — user_id, value_mgdl, source (`(manual)` | `(device)`), recorded_at

- `medications` — `user_id`, `name`, `dosage`, `schedule` (times per day), `created_at`
- `medication_logs` — `medication_id`, `scheduled_for`, `taken_at` (nullable), `status` (`taken` | `missed` | `pending`)
- `meals` — `user_id`, `description`, `photo_url` (nullable), `estimated_carbs_g`, `logged_at`
- `activity_logs` — `user_id`, `type`, `duration_minutes`, `logged_at`
- `alerts` — `user_id`, `type` (`hypo_risk` | `hyper_risk` | `missed_dose`), `payload` (jsonb), `created_at`, `acknowledged_at`
- `devices` — `user_id`, `device_type` (`glucometer` | `cgm` | `wearable`), `external_id`, `paired_at`, `last_synced_at`

Definition of Done:

- Migrations run cleanly against a fresh database: `docker-compose exec api npm run migrate`.
 - A seed script creates one patient, one clinician, and a `care_link` between them, plus a week of sample glucose readings — run it and query the DB to confirm rows exist.
 - Re-running migrations on an already-migrated DB is a no-op (idempotent).
-

Phase 2 — Authentication & Authorization

Goal: Every subsequent endpoint can trust `req.user.id` and `req.user.role`.

1. `POST /auth/register`, `POST /auth/login` (issues access + refresh JWT), `POST /auth/refresh`, `POST /auth/logout` (invalidate refresh token, store denylist in Redis).
2. Password hashing with bcrypt (cost factor 12).
3. Middleware that verifies the JWT and attaches `req.user`; a second middleware factory `requireRole(["clinician"])` for role-gated routes.
4. Care-data access rule used everywhere from here on: a clinician/caregiver may only read a patient's data if an **active** `care_links` row exists between them. Build this as a single reusable authorization helper, not copy-pasted per route.

Definition of Done:

- Register → login → call a protected test route with the access token → 200. Same call with no token or an expired token → 401.
 - A clinician without an active `care_link` to a patient gets 403 when requesting that patient's data; after activating the link, the same request succeeds.
 - Automated test file covering all four cases above passes in CI (`npm test`).
-

Phase 3 — Core Patient APIs

Goal: FR-1 through FR-5 from the PRD — the CRUD that the mobile/web app actually calls day to day.

Endpoints (all scoped to the authenticated patient, or a linked clinician/caregiver reading with permission):

- `POST/GET /glucose-readings` (manual entry + list, paginated, filterable by date range)
- `POST/GET/PATCH /medications`, `POST /medications/:id/log` (mark a scheduled dose taken/missed)
- `POST/GET /meals`
- `POST/GET /activity-logs`
- `GET /dashboard/summary` — one aggregate endpoint returning current reading, time-in-range %, today's medication status, and latest alert, so the frontend doesn't need five separate calls on load.

Definition of Done:

- Postman/Supertest collection exercises every endpoint's happy path and at least one validation failure (e.g., glucose value out of plausible range 20–600 mg/dL should 400, not 500).
- `GET /dashboard/summary` returns correct aggregates against the seeded data from Phase 1 — check the numbers by hand against the seed values.
- Load the web frontend (if available) against this API and confirm the Overview screen renders real data instead of its mock data.

Phase 4 — IoT Device Integration Layer

Goal: FR-7 — automated glucose ingestion from glucometers/CGMs, without the app needing to poll constantly.

1. `POST /devices/pair` — registers a device against a user (`external_id` from the glucometer/CGM SDK).
2. `POST /devices/:id/readings` — ingestion endpoint the device gateway (a phone acting as a Bluetooth bridge, per the PRD's architecture) pushes readings to; write into `glucose_readings` with `source = 'device'`.
3. Idempotency: ingesting the same reading twice (same device + timestamp) must not create a duplicate row — enforce with a unique constraint, not just application logic.
4. On each new reading, publish an event on Redis pub/sub (`glucose.new_reading`) — Phase 6 (alerts) and Phase 7 (real-time push) both subscribe to this instead of polling the database.

Definition of Done:

- Simulate a device push with a script that posts 10 readings, including one deliberate duplicate — confirm exactly 9 rows land in the table.
 - Confirm the Redis event fires by subscribing a throwaway script to `glucose.new_reading` and watching it print events as readings come in.
 - Document the actual glucometer/CGM SDK integration point as a stub/interface (`DeviceGateway`) rather than hardcoding one vendor, since the PRD scopes MVP to "one well-documented glucometer SDK" but the interface should not block adding a second later.
-

Phase 5 — ML Prediction Service (glycemic forecasting)

Goal: FR-6 — the LSTM/Random Forest forecasting described in the PRD, served as a real endpoint the API can call.

In `/ml-service`:

1. Training pipeline: script that loads the PIMA dataset (or the seeded synthetic time series from Phase 1 for local dev), trains an LSTM for short-horizon glucose forecasting, and a Random Forest classifier for hypo/hyperglycemia risk, saving both to `/ml-service/models/`.
2. `POST /predict/forecast` — takes a recent glucose window (+ optionally recent meals/activity) and returns predicted values for the next 30/60/90/120/150/180 minutes with a confidence interval, matching the shape the frontend's Trends chart expects (`t`, `predicted`, `low`, `high`).
3. `POST /predict/risk` — returns a probability of a hypo or hyper event in the next N hours, plus a plain-language reason string if feasible (e.g., "recent dinner + declining trend").
4. `api` calls these two endpoints server-to-server (never expose `ml-service` directly to the internet) and writes a row to `alerts` when risk crosses a threshold.
5. Version the model artifact (filename includes a date or hash) so retraining doesn't silently break a running service — `ml-service` should log which model version answered each request.

Definition of Done:

- Training script runs end-to-end and reports validation accuracy/MAE — record the numbers in `/docs/model-notes.md` so future retraining has a baseline to beat.
- `curl` the `/predict/forecast` endpoint with a sample window and confirm the response shape matches what the Trends screen renders (compare against the mock `forecast` array in the frontend).

- Feed in a deliberately rising glucose pattern and confirm `/predict/risk` returns a meaningfully higher probability than a flat/stable pattern — a hardcoded constant response would pass a shape check but fail this.
-

Phase 6 — NLP Dietary Analysis Service

Goal: FR-4 — turn a logged meal description into a carbohydrate estimate and personalized feedback.

1. `POST /predict/meal-analysis` in `ml-service` — accepts free-text meal description (and optionally a photo URL, out of scope for MVP per the PRD unless time allows), returns estimated carbs, a rough macro breakdown, and a one-line recommendation.
2. Start with a rules/lookup-table + a food-nutrition dataset approach if a full NLP model is too heavy for the timeline; note this simplification explicitly in `/docs/model-notes.md` so it's not mistaken for the final approach.
3. `api`'s `POST /meals` endpoint calls this service and stores the estimated carbs alongside the raw text.

Definition of Done:

- Log meals with varied phrasing ("2 rotis with dal", "chicken salad, no dressing") and confirm carb estimates are in a plausible range, not identical for every input.
 - Confirm the Diet screen's meal list (from the frontend) can be populated entirely from this pipeline's output.
-

Phase 7 — Real-Time Alerts & Notifications

Goal: FR-9 — alerts actually reach the patient and clinician promptly, not just get written to a table.

1. WebSocket gateway in `api`: authenticated clients (patient, linked clinician/caregiver) subscribe to their own channel.
2. Subscribe to the Redis events from Phase 4 and the alert-writes from Phase 5; push to the relevant WebSocket channel(s) immediately.
3. Missed-dose detection: a scheduled job (node-cron or a queue) checks `medication_logs` for entries past their scheduled time with no `taken_at`, marks them `missed`, and raises an `alerts` row.
4. Fallback for offline clients: alerts are still queryable via `GET /alerts` so a client that reconnects later isn't missing anything.

Definition of Done:

- Open a WebSocket test client, trigger a Phase 4 device reading that crosses the risk threshold, and confirm the alert arrives over the socket within a couple of seconds.
 - Stop the client, generate an alert, reconnect, and confirm `GET /alerts` shows it (nothing is silently dropped for offline clients).
 - Let a scheduled dose's time pass without logging it and confirm the missed-dose job picks it up on its next run.
-

Phase 8 — Clinician Dashboard & Data Sharing APIs

Goal: FR-8 — everything the clinician-facing screens (Care Team / dashboard) need, properly scoped by the Phase 2 care-link rule.

1. `GET /clinician/patients` — list of patients linked to the authenticated clinician, with a quick-glance status (time-in-range, open alerts).
2. `GET /clinician/patients/:id/report` — the detailed view (glucose history, medication adherence, meal log) for one patient, enforcing the care-link check from Phase 2.
3. Messaging: `POST /messages`, `GET /messages?withUserId=` — simple patient↔clinician thread, also pushed over the Phase 7 WebSocket for real-time delivery.
4. `POST /care-links/:id/revoke` — patient can revoke a clinician/caregiver's access at any time; revoked links must immediately stop passing the Phase 2 authorization check (don't rely on a cache that could serve stale access).

Definition of Done:

- As a clinician, list patients, open one's report, and confirm the numbers match that patient's own `/dashboard/summary` from Phase 3.
 - Revoke a care link and immediately re-request the report as that clinician — confirm it now 403s.
 - Send a message from patient to clinician and confirm it's delivered both via `GET /messages` and the live WebSocket if the recipient is connected.
-

Phase 9 — Security Hardening & Test Coverage

Goal: Meet the PRD's non-functional requirements before anything ships.

1. Encrypt sensitive fields at rest where appropriate (at minimum, ensure the DB connection uses TLS and backups are encrypted); confirm HTTPS is enforced in front of both services (a reverse proxy like Caddy/Nginx in the compose file, or documented for production).
2. Rate limiting on `/auth/*` and on the device ingestion endpoint (prevent a misbehaving device from flooding the DB).

3. Input validation on every endpoint (schema validation library — Zod for Node, Pydantic for FastAPI) — reject malformed payloads before they hit business logic.
4. Full test suite: unit tests per module, integration tests per phase's Definition of Done turned into permanent automated tests, and a basic load test (e.g., k6 or autocannon) against `/dashboard/summary` and `/glucose-readings` to confirm the NFR of syncing within a few seconds holds under light concurrent load.
5. Dependency and container vulnerability scan (`npm audit`, `pip-audit`, or Docker's built-in scan) with no unresolved criticals.

Definition of Done:

- Full test suite passes with `docker-compose run api npm test` and `docker-compose run ml-service pytest`.
 - Rate-limit test: hammer `/auth/login` past the limit and confirm a 429, not a crash.
 - Load test report saved to `/docs/load-test-results.md` with actual latency numbers, not just "it worked."
-

Phase 10 — Deployment & Monitoring

Goal: A reproducible way to run this outside a developer's laptop.

1. Production Dockerfiles (multi-stage builds, no dev dependencies in the final image) for `api` and `ml-service`.
2. CI pipeline (GitHub Actions or equivalent) that runs the Phase 9 test suite on every push and blocks merge on failure.
3. Structured logging (JSON logs) from both services, plus a `/metrics` endpoint or equivalent for basic request-rate/error-rate/latency monitoring.
4. Document environment variables and deployment steps in `/docs/deployment.md` — a fresh machine should be able to follow it and get a working instance without guessing.

Definition of Done:

- CI pipeline runs end-to-end on a test push, including a deliberately broken test to confirm it actually fails the build (then revert the break).
 - Following `/docs/deployment.md` from scratch (e.g., in a clean container or VM) results in a running system whose `/health` endpoints respond — this is the real acceptance test for the whole brief.
-

Notes for the agent

- Treat each phase's Definition of Done as a gate, not a suggestion — run the commands, read the output, and only report a phase complete once they pass.

- Keep `/docs/model-notes.md`, `/docs/load-test-results.md`, and `/docs/deployment.md` updated as you go; they're cheap now and expensive to reconstruct later.
- The frontend prototypes (mobile app and web dashboard) already define the exact data shapes each screen expects (glucose history arrays, forecast arrays with `t/predicted/low/high`, medication objects, message threads) — match those shapes in the API responses so integration doesn't need a translation layer.
- If a step in a later phase reveals that an earlier phase's schema or contract was wrong, fix it via a new migration/version, not a silent workaround — note the change in the relevant `/docs` file.